

TecNM Campus Matehuala

Ingeniería de Software

2.3

Diseño Arquitectónico

TallerTec — Arquitectura del Sistema

Proyecto: TallerTec

Sistema Integral de Gestión de Talleres Deportivos

Equipo de Desarrollo: Virgilio Power | Febrero 2025

1. Introducción

Este documento describe la arquitectura de software de TallerTec a nivel de decisiones de diseño, patrones aplicados, flujos de datos entre capas y especificaciones técnicas de cada componente. Sirve como guía maestra para que el equipo de desarrollo implemente el sistema con consistencia y sin ambigüedades.

La arquitectura de TallerTec sigue el patrón de **tres capas (Three-Tier Architecture)** adaptado a una arquitectura BaaS (Backend-as-a-Service) con Next.js 14 como framework full-stack y Supabase como plataforma de datos y autenticación.

2. Vista General de la Arquitectura

2.1 Principios Arquitectónicos

- Separación de responsabilidades: Presentación, Lógica y Datos completamente desacopladas.
- Seguridad en profundidad: Autenticación en Supabase Auth, autorización por roles en API Routes, RLS en BD.
- Mobile-first: PWA como interfaz principal de estudiantes, con Service Worker para modo offline.
- Zero-cost deploy: 100% en tiers gratuitos durante la fase académica y piloto.
- Escalabilidad horizontal: Vercel Edge Functions + Supabase Pro cuando se requiera escalar.

2.2 Diagrama de Arquitectura

Figura 2.1 — Diagrama de Arquitectura General TallerTec

2.3 Capas del Sistema

Capa	Tecnología	Responsabilidad
Presentación	Next.js 14 + Tailwind CSS	Interfaz de usuario, routing, componentes React, Service Worker
Lógica de Negocio	Next.js API Routes + Zod	Validación, reglas de negocio, orquestación de servicios
Datos	Supabase (PostgreSQL + Auth + Storage)	Persistencia, autenticación, archivos, eventos en tiempo real

3. Capa de Presentación

3.1 Tres Interfaces de Usuario

La capa de presentación consta de tres interfaces diferenciadas, cada una optimizada para su actor y dispositivo:

3.1.1 TallerTec App — PWA Estudiantes

Aspecto	Especificación
Tipo	Progressive Web App (PWA) instalable
Dispositivo objetivo	Móvil Android/iOS (320px–768px)
Rutas principales	/ (catálogo), /mis-talleres, /mi-qr, /mi-progreso, /constancias
Características PWA	manifest.json, Service Worker (next-pwa), cache offline (horas y QR)
Flujos clave	Inscripción (3 pasos), Ver QR (1 paso), Solicitar constancia (2 pasos)
Autenticación	Persistida en localStorage con token JWT de Supabase

3.1.2 TallerTec Admin — Panel Responsables

Aspecto	Especificación
Tipo	Web app responsive (no PWA)
Dispositivo objetivo	Móvil con cámara (responsable escanea QR)
Rutas principales	/responsable/asistencia, /responsable/inscritos, /responsable/estadisticas
Función crítica	Escáner QR (html5-qrcode) con activación de cámara trasera
Realtime	Supabase Realtime — lista de asistencia actualiza en vivo

3.1.3 TallerTec Dashboard — Oficina de Deportes

Aspecto	Especificación
Tipo	Web app con dashboard de datos
Dispositivo objetivo	Desktop / Tablet (768px+)
Rutas principales	/admin/dashboard, /admin/talleres, /admin/constancias, /admin/reportes
Gráficas	Recharts: Líneas (asistencia por semana), barras (talleres), donut (estados)
Exportación	Módulo xlsx.js para Excel (.xlsx) y @react-pdf/renderer para reportes PDF

3.2 Protección de Rutas — Middleware

El archivo `middleware.ts` intercepta todas las peticiones y aplica las siguientes reglas de acceso:

Patrón de Ruta	Regla de Acceso
/api/auth/**	Pública — sin autenticación requerida
/	Pública — landing page
/catalogo	Requiere: rol ESTUDIANTE
/responsable/**	Requiere: rol RESPONSABLE_TALLER

Patrón de Ruta	Regla de Acceso
/admin/**	Requiere: rol ADMIN_OFICINA
/api/talleres (GET)	Requiere: cualquier rol autenticado
/api/constancias/aprobar	Requiere: rol ADMIN_OFICINA

4. Capa de Lógica de Negocio

4.1 API Routes — Especificación Completa

Todas las rutas de API se encuentran bajo `/app/api/` y siguen el patrón REST. La lógica de negocio NUNCA se ejecuta en el cliente — siempre en el servidor.

Endpoint	Método	Rol Requerido	Descripción
/api/auth/register	POST	Público	Crear cuenta con email @matehuala.tecnm.mx
/api/auth/login	POST	Público	Autenticar y retornar JWT
/api/auth/logout	POST	Autenticado	Invalidar sesión
/api/talleres	GET	Autenticado	Listar talleres activos con cupos
/api/talleres	POST	ADMIN_OFICINA	Crear nuevo taller
/api/talleres/:id	PUT	ADMIN_OFICINA	Actualizar taller
/api/talleres/:id	DELETE	ADMIN_OFICINA	Desactivar taller (soft delete)
/api/inscripciones	POST	ESTUDIANTE	Inscribirse a taller
/api/inscripciones/:id	DELETE	ESTUDIANTE	Darse de baja
/api/inscripciones/manual	POST	RESPONSABLE_TALLER	Inscripción tardía
/api/asistencias	POST	RESPONSABLE_TALLER	Registrar asistencia (QR o manual)
/api/asistencias/:taller_id	GET	RESPONSABLE_TALLER	Historial de asistencias
/api/horas/me	GET	ESTUDIANTE	Mis horas acumuladas + progreso
/api/horas/:estudiante_id	GET	ADMIN_OFICINA	Horas de cualquier estudiante
/api/constancias	POST	ESTUDIANTE	Solicitar constancia
/api/constancias/:id/aprobar	PUT	ADMIN_OFICINA	Aprobar y generar PDF
/api/constancias/:id/rechazar	PUT	ADMIN_OFICINA	Rechazar con observaciones
/api/constancias/:id/download	GET	ESTUDIANTE	Descargar PDF propio

Endpoint	Método	Rol Requerido	Descripción
/api/reportes/dashboard	GET	ADMIN_OFICINA	Métricas globales del programa
/api/reportes/excel	GET	ADMIN_OFICINA	Exportar Excel de inscritos/horas
/api/periodos	GET	ADMIN_OFICINA	Listar periodos
/api/periodos	POST	ADMIN_OFICINA	Crear período semestral

4.2 Reglas de Negocio Críticas

Regla	Implementación Técnica
Email debe ser @matehuala.tecnm.mx	Regex en Zod schema: z.string().regex(/@matehuala\.tecnm\.mx\$/)
Un estudiante — un taller por período	UNIQUE constraint (estudiante_id, taller_id, periodo_id) en PostgreSQL
Cupo no puede ser negativo	CHECK constraint cupo_disponible >= 0 + transacción atómica
Solo solicitar constancia con ≥20 hrs	API valida: SELECT horas_acumuladas FROM inscripciones WHERE estudiante_id=X
Solo admin genera constancias oficiales	RLS policy: USING (auth.uid() IN (SELECT id FROM usuarios WHERE rol='ADMIN_OFICINA'))
Horas calculadas automáticamente	Trigger PostgreSQL: UPDATE inscripciones SET horas_acumuladas = SUM(asistencias.horas_computadas)
Folio único de constancia	formato: TALL-AÑO-SEMESTRE-SECUENCIAL (ej: TALL-2025-1-00047)
Sesión expira en 7 días	Configurado en Supabase Auth: JWT expiry = 604800 segundos

4.3 Generación de PDF de Constancia

El proceso de generación de PDF se ejecuta en una Supabase Edge Function para evitar timeouts en Next.js. Usa [@react-pdf/renderer](#) compilado para Deno:

1. Admin aprueba solicitud desde dashboard.
2. API Route /api/constancias/:id/aprobar actualiza estado a APROBADA.
3. Edge Function gen-constancia-pdf es invocada con el ID de constancia.
4. Edge Function consulta datos del estudiante, taller, horas y desglose de asistencias.
5. Genera PDF con: logo TecNM, datos del estudiante, folio único, tabla de asistencias, firma digital del coordinador.
6. Sube PDF a Supabase Storage bucket constancias-pdf con ruta: {estudiante_id}/{folio}.pdf.
7. Actualiza registro de constancia con archivo_url y estado = GENERADA.
8. Envía notificación por email al estudiante via Resend API.

5. Capa de Datos — Supabase

5.1 Row Level Security (RLS) — Políticas por Tabla

Tabla	Operación	Política RLS
usuarios	SELECT	<code>auth.uid() = id OR rol='ADMIN_OFICINA'</code>
usuarios	UPDATE	<code>auth.uid() = id</code> (solo perfil propio)
talleres	SELECT	<code>activo = true</code> (todos autenticados)
talleres	INSERT/UPDATE	<code>rol = 'ADMIN_OFICINA'</code>
inscripciones	SELECT	<code>estudiante_id = auth.uid() OR responsable_del_taller OR admin</code>
inscripciones	INSERT	<code>rol = 'ESTUDIANTE' AND cupo_disponible > 0</code>
asistencias	SELECT	<code>inscripcion.estudiante_id = auth.uid() OR responsable OR admin</code>
asistencias	INSERT	<code>rol = 'RESPONSABLE_TALLER' AND es_responsable_del_taller</code>
constancias	SELECT	<code>estudiante_id = auth.uid() OR rol = 'ADMIN_OFICINA'</code>
constancias	INSERT/UPDATE	<code>rol = 'ADMIN_OFICINA'</code>
notificaciones	SELECT	<code>usuario_id = auth.uid()</code>

5.2 Triggers y Funciones PostgreSQL

Trigger / Función	Descripción
<code>calcular_horas_acumuladas()</code>	AFTER INSERT en asistencias: recalcula horas_acumuladas en inscripciones
<code>verificar_meta_20h()</code>	AFTER UPDATE en inscripciones: si horas >= 20, crea notificación y actualiza estado=COMPLETADA
<code>handle_new_user()</code>	AFTER INSERT en auth.users: crea registro en tabla usuarios con rol=ESTUDIANTE
<code>generar_folio_constancia()</code>	BEFORE INSERT en constancias: genera folio único TALL-AÑO-SEM-SEQ
<code>update_cupo_disponible()</code>	AFTER INSERT/DELETE en inscripciones: actualiza cupo_disponible en talleres

5.3 Índices de Rendimiento

Índice	Justificación
<code>idx_inscripciones_estudiante</code>	Búsquedas frecuentes: horas del estudiante
<code>idx_inscripciones_taller</code>	Búsquedas: inscritos en un taller

Índice	Justificación
<code>idx_asistencias_inscripcion</code>	JOIN frecuente: historial de asistencias
<code>idx_talleres_periodo_activo</code>	Catálogo: filtro más común
<code>idx_constancias_estudiante</code>	Verificar constancias pendientes del estudiante
<code>idx_usuarios_email (UNIQUE)</code>	Login: búsqueda por email
<code>idx_usuarios_qr (UNIQUE)</code>	Validación QR: búsqueda por codigo_qr

6. Flujos de Datos Completos

6.1 Flujo de Autenticación y Sesión

- Estudiante ingresa email (@matehuala.tecnm.mx) y contraseña en la pantalla de login.
- PWA llama a `supabase.auth.signInWithPassword({ email, password })`.
- Supabase Auth verifica credenciales y retorna `access_token` (JWT 7 días) + `refresh_token`.
- Next.js middleware intercepta la petición y verifica el JWT en cada request protegido.
- El rol del usuario se extrae del JWT (claims personalizados agregados por trigger `handle_new_user`).
- Redirección automática al dashboard correcto según el rol.

6.2 Flujo de Modo Offline (PWA)

El Service Worker cachea las siguientes recursos para funcionamiento sin red:

Recurso Cacheado	Estrategia
Página /mi-qr	Cache First – el QR nunca cambia una vez generado
Página /mi-progreso	Stale While Revalidate – muestra cache, actualiza en fondo
Assets estáticos (JS/CSS)	Cache First con versionado
API /api/horas/me	Network First – intenta red, fallback a cache

7. Seguridad — Especificación Completa

Capa	Mecanismo	Implementación
Transporte	HTTPS TLS 1.2+	Automático en Vercel + Supabase
Autenticación	JWT HS256	Supabase Auth, expiración 7 días
Autorización	RLS + Middleware	Política por tabla + verificación de rol en API Route
Contraseñas	bcrypt salt=12	Gestionado internamente por Supabase Auth

Capa	Mecanismo	Implementación
Inputs	Zod schemas	Validación estricta en TODOS los endpoints
CORS	Next.js config	Origen permitido: solo dominio de producción
Archivos PDF	Storage privado	URL firmada con expiración 1 hora para descargas
Rate Limiting	Vercel Edge	100 req/min por IP en endpoints sensibles
Variables secretas	Env vars Vercel	Nunca en código fuente. Acceso server-side únicamente

8. Requisitos No Funcionales — Diseño de Cumplimiento

RNF	Meta	Estrategia de Diseño
RNF-REND-01	<2 seg	SSR en Next.js + Vercel Edge CDN + compresión gzip automática
RNF-REND-02	<1 seg QR	html5-qrcode procesa frames localmente sin red
RNF-REND-04	<200ms API	Índices PostgreSQL + Supabase connection pooling
RNF-REND-05	100 users	Vercel Edge (sin estado) + Supabase connection pooler (PgBouncer)
RNF-DISP-02	Offline QR+horas	Service Worker + Cache API en PWA
RNF-DISP-03	99% uptime	Vercel SLA 99.99% + Supabase SLA 99.9%
RNF-USA-01	Responsive	Tailwind CSS mobile-first, breakpoints 375/768/1280
RNF-SEG-03	RLS	Políticas habilitadas en todas las tablas desde migración inicial